

FinTech and AI in Finance

Class 1: Financial Data and AI-Assisted Coding with Codex and Python

Gavin Feng

Semester A, 2026/27

City University of Hong Kong

AI in Finance and FinTech: Two Course Blocks

Layer	Typical AI applications	EF5560 emphasis
Information and text	Market, accounting, macro, transaction, filing, news, and rule data.	Class 1 data discipline; Class 12 LLM text applications.
Prediction	Returns, risk, default, fraud, demand, and customer behavior.	Core: regression and linear/nonlinear ML in Classes 2–4.
Decisions and products	Portfolios, prices, limits, payments, lending, advice, and insurance.	Class 5 investment decisions; Classes 10–12 product cases.
Infrastructure and control	APIs, cloud, blockchain, cybersecurity, RegTech, and model governance.	Mechanisms and controls in Classes 6–12.

Part I: ML in Finance (Classes 1–5)

Data, return prediction, and portfolio evaluation

Part II: FinTech (Classes 6–12)

Digital assets, payments, lending, advice, cybersecurity, and RegTech

The two parts share a financial setting but address different questions.

A Financial AI Product Needs More Than a Model

Layer	Design question	Failure if ignored
Data and access	Is the information timely, point-in-time, permitted, and usable by the model?	Leakage, stale inputs, or data that cannot be deployed.
Model and benchmark	Is the task prediction or generation, and what simple alternative must it beat?	Plausible output without measurable decision value.
Decision process and distribution	Will users receive an API, copilot, automated rule, or human-reviewed recommendation?	A strong model that never changes a financial decision.
Controls and accountability	Who validates, monitors, secures, overrides, and audits the system?	Model risk, unclear responsibility, and costly operational failures.

Keep in mind

Financial value comes from the full system: data, model, decision process, distribution, and controls.

Two Ways to Build and Distribute AI



Sam Altman

CEO, OpenAI

Approach: connect frontier research to consumer products, developer APIs, and enterprise deployment on centrally operated infrastructure.

Questions for finance users: How much do model quality and distribution help, and how dependent is the product on compute, pricing, access, and governance?



Liang Wenfeng (梁文锋)

Founder, DeepSeek; co-founder, High-Flyer

Approach: move from quantitative research into foundation models and distribute DeepSeek-R1 code and weights under the MIT License.

Questions for finance users: How much do open weights and engineering efficiency help, and who manages commercialization, operations, and model risk?

Keep in mind

Which layer a firm controls decides what it can charge for. Ask that before asking whose model is better.

Sources: [OpenAI, structure](#); [DeepSeek-AI, R1 repository](#); [Hangzhou Municipal Government, Liang profile](#). Portraits are AI-generated editorial illustrations, not photographs.

How the Course Is Assessed

Assessment	Weight
FinTech company case report	10%
Recorded video presentation	20%
ML investment assignment	20%
Participation and in-class work	10%
Two-hour final examination	40%
Total	100%

Key milestones

- ▶ ML assignment: teams of up to three; released after Class 5 and due at the end of Week 7.
- ▶ Case report and recorded video: due in Week 12.
- ▶ Video: one MP4 file, no more than 10 minutes or 80 MB.
- ▶ Every member speaks for at least two minutes; there is no in-person case presentation.

Keep in mind

Canvas is the source of truth for rubrics, submission links, and any administrative updates.

AI-Assisted Coding in This Course

By the end of this class, you should be able to:

1. define a financial-data task before asking an AI assistant for code;
2. verify a return table using its prices, dates, rows, and source; and
3. distinguish a date shown in a file from the date information becomes usable for prediction.

Keep in mind

Today is about financial judgement around data and AI tools, not memorising Python syntax.

What AI-Assisted Coding Means in This Course

Describe the task in plain language, let an AI assistant write and run a small Python job, inspect the result, and revise until the output passes explicit checks.

The AI can help with

- ▶ translating a financial task into executable Python;
- ▶ explaining unfamiliar syntax; and
- ▶ revising code after an error.

You still decide

- ▶ the target, sample, variables, and timing;
- ▶ what was observable at each date; and
- ▶ whether the output is economically sensible.

Keep in mind

Give an instruction → run Python → inspect → revise → save. In class, we work on the request and the evidence rather than typing code line by line.

PDD-JD: A Relative-Value Trade

PDD Holdings (PDD)

PDD's reported platforms are Pinduoduo and Temu. The investment thesis treats PDD as outside the food-delivery competition.

JD.com (JD)

JD officially launched Food Delivery in February 2025. The new business adds investment and execution risk, but could also raise customer traffic and cross-selling.

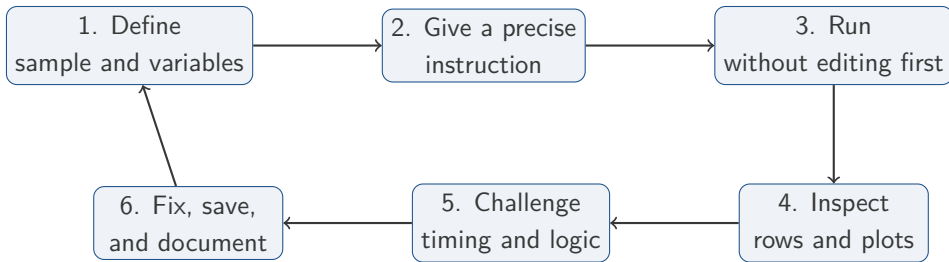
The question

Can a dollar-neutral long PDD / short JD position benefit if investors price the new food-delivery exposure more heavily into JD?

Keep in mind

The position direction comes from a business hypothesis, not a price z-score. Historical data must test the hypothesis.

A Six-Step Procedure



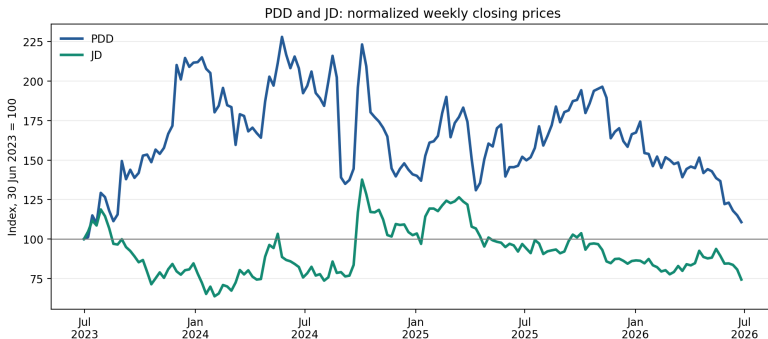
The loop is complete only when the output passes explicit checks.

Specify the Data before Requesting Code

Give the coding assistant all six specifications:

1. **Language and packages:** R with `quantmod`, or Python with `yfinance` and `pandas`.
2. **Data:** ticker, source, start date, end date, and frequency.
3. **Price convention:** adjusted or unadjusted.
4. **Forecast clock:** when the signal is formed and which future return is the target.
5. **Predictors:** exact formulas and rolling windows.
6. **Outputs and checks:** CSV files, plots, dimensions, missing counts, and sample rows.

PDD and JD: Relative Price Paths



Keep in mind

Both series start at 100 on 30 June 2023. By 26 June 2026, PDD is at 110.7 and JD is at 74.4. Their relative price did not remain stable.

Source: Nasdaq daily closing prices, 26 June 2023–26 June 2026. This is a price comparison, not a total-return comparison.

Let Codex Install R and Python—With Your Approval

Example instruction for Codex

I use [Windows 11 / macOS]. Check whether Python 3 and R already work. If either is missing, use the official VS Code or RStudio installer. Before any installation, show the package, destination, and command; then wait for approval. Install yfinance, pandas, matplotlib, akshare, quantmod, and TTR, and report the installed versions.

Four steps

1. **Inspect** what already works.
2. **Review** the source and destination.
3. **Approve** the installation.
4. **Verify** versions and imports.

Pause if the scope changes

Do not provide credentials or unrelated personal files.

Today's Samples, Variables, and Dates



Keep in mind

Markets and macro share one 240-month calendar. The later HSTECH 30 exercise uses denser weekly observations, and every forecasting row uses only information known before its return is realized.

Free data support teaching. Research needs point-in-time data with documented revisions and historical constituents.

Use Codex to Construct 156 Weekly Returns

Example instruction

Using Python and adjusted AAPL prices, build Friday-labelled weekly prices from 30 June 2023 through 26 June 2026 and simple weekly returns from 7 July 2023 through 26 June 2026. Download enough earlier data to calculate the first return. Run the job, save the table, and report the first date, last date, row count, and missing values.

Fix before the run

- ▶ adjusted-price convention;
- ▶ Friday week labels; and
- ▶ exact calendar endpoints.

Accept only if Python returns

- ▶ 157 weekly prices;
- ▶ 156 non-missing returns; and
- ▶ a saved, date-sorted file.

Keep in mind

The classroom task is to specify the sample, variables, dates, and checks; Codex handles the Python execution.

Long PDD / Short JD Position

Long PDD

The thesis gives PDD a relative advantage because it does not carry JD's direct food-delivery investment and execution exposure.

Short JD

The short leg offsets some China-consumer news while retaining exposure to the relative difference between the two businesses.

Keep in mind

The position is dollar neutral, but it does not neutralize all China-consumer, currency, regulatory, or business risk.

At Friday close t , set equal dollar weights:

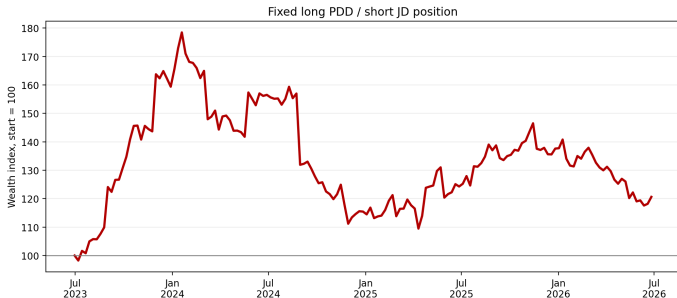
$$w_t^{PDD} = +0.5, \quad w_t^{JD} = -0.5.$$

The following week's portfolio return is

$$r_{t+1}^{LS} = 0.5r_{t+1}^{PDD} - 0.5r_{t+1}^{JD}.$$

Rebalance to the same dollar weights each Friday.

Long PDD / Short JD: Historical Illustration



Long leg

PDD: +50%

Rebalanced weekly

Short leg

JD: -50%

156 weekly returns

Result

7 Jul 2023–26 Jun 2026

Cumulative return: +20.6%

Keep in mind

The window starts before JD Food Delivery launched. It illustrates long-short return arithmetic, but cannot establish that food delivery caused the result.

Let Codex Call Two Market-Data Sources

```
import akshare as ak
import pandas as pd
import yfinance as yf

global_px = yf.download(
    ["^GSPC", "^HSI"],
    start="2006-06-01", end="2026-07-01",
    auto_adjust=True, progress=False
)["Close"]

csi = ak.stock_zh_index_daily_em(
    symbol="sh000300", start_date="20060601",
    end_date="20260630"
)
csi["date"] = pd.to_datetime(csi["date"])
csi_px = csi.set_index("date")["close"]
csi_px.name = "CSI300"
daily = global_px.join(csi_px, how="outer")
```

- ▶ Each source returns daily closes.
- ▶ join aligns observations by date.
- ▶ how="outer" keeps trading days from either market.

Use the strongest available source for each series; one website need not supply the whole table.

Use Codex to Check the 240-Month Table

```
monthly = daily.resample("ME").last()  
monthly = monthly.loc["2006-07":"2026-06"]
```

```
print(monthly.index.min())  
print(monthly.index.max())  
print(monthly.shape)  
print(monthly.isna().sum())
```

Verified output

- ▶ first month: 2006-07-31
- ▶ last month: 2026-06-30
- ▶ shape: (240, 3)
- ▶ missing months: zero

- ▶ Month-end labels create a common calendar even when local trading days differ.
- ▶ These are price-index returns; dividends are not included.

Keep in mind

The shared calendar lines up the dates. Holidays, trading hours, and index construction rules still differ across the three markets.

State the Return Definition, Then Ask for the Plot

Tell Codex exactly what to calculate

From the checked weekly adjusted prices, calculate $r_t = P_t/P_{t-1} - 1$. Run Python and return a date-indexed table plus one price chart and one return chart. Do not fill the first missing return.

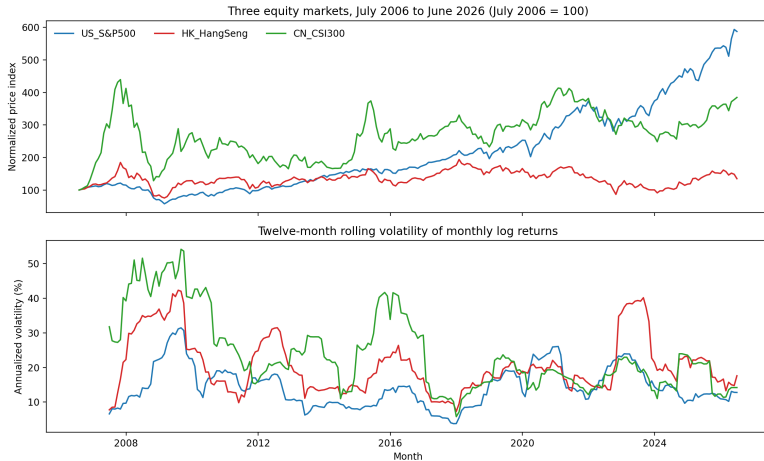
Inspect the table

- ▶ prices and returns share the same week labels;
- ▶ the first return is missing by construction; and
- ▶ no weekend or duplicate rows appear.

Challenge the plots

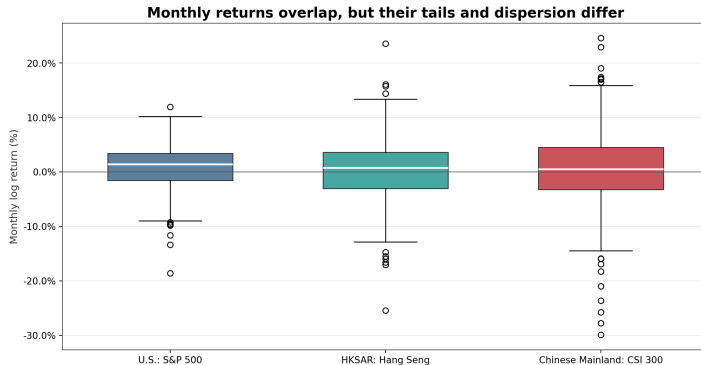
- ▶ adjusted prices should not show a mechanical split crash;
- ▶ returns should fluctuate around a much smaller scale; and
- ▶ extreme weeks should be traced to dates, not deleted automatically.

Twenty Years of Market Data Fit One Common Clock



Source: Yahoo Finance through `yfinance`; 240 month-end observations, 2006-07 through 2026-06, downloaded July 2026.

Monthly Returns Differ across the Three Markets



Keep in mind

When a return looks extreme, check the date and the price adjustment before you touch the observation.

Adjusted and Unadjusted Prices Answer Different Questions

- ▶ A stock split changes the quoted price without creating an investor loss.
- ▶ A cash dividend creates a return even though the ex-dividend price falls.
- ▶ An adjusted series tries to place prices on a comparable economic basis.
- ▶ Package defaults and vendor conventions can change; make the choice explicit.

Class discussion

What would a 2-for-1 split do to a return computed from unadjusted closing prices?

Ask Codex for Four Checks before Any Calculation

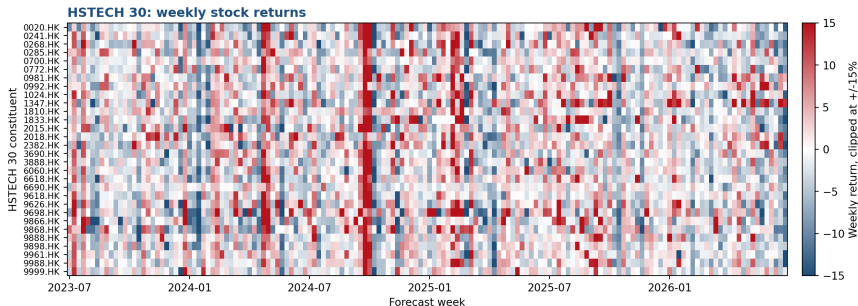
Data-checking instruction

Before calculating returns, run Python and show: (1) table dimensions, (2) exact column names and types, (3) the first and last three dated rows, and (4) missing-value and descriptive-statistic summaries.

Reading the output is the part that cannot be delegated. Two questions decide whether to continue: which column is the adjusted economic price, and are the missing observations and magnitudes plausible for this market?

The Individual-Stock Exercise Uses 156 Weekly Cross-Sections

July 2023 membership, 156 weekly cross-sections through June 2026



Keep in mind

Read the heatmap: each row is one stock, each column is one forecast week, and colour is the weekly return. The July 2023 HSTECH 30 membership remains fixed throughout the 156 weeks.

Multiple-Choice Question: What Does a Frozen Universe Protect?

A researcher starts with the July 2023 HSTECH 30 constituents. When one stock delists, it is replaced by a company that joined the index in 2025 so that every week still has 30 stocks. What is the main problem?

- A) Future membership information changes the historical investment universe.
- B) Weekly returns can no longer be computed for any surviving stock.
- C) The replacement automatically removes all extreme returns.
- D) The market index must become equally weighted.

Discuss

What should the frozen panel record after the original constituent delists, and why?

Add Macro Data and Company Financials

From an Economic Event to Usable Information

Economic period → File date → Release or filing → Usable to forecast

Data	What the visible date means	Use only when
Market index	trading month or day	the decision uses only earlier closes
Daily rates	trading day	aggregated, then lagged for the forecast month
Monthly macro	reference month	the release is public
Company financials	fiscal period end	the 10-Q or 10-K is filed

Keep in mind

A date in a file is not automatically a date when investors could have known the value.

Tell the AI Which Macro Series You Mean

Macro-data instruction

Download FRED series DGS10, DFF, UNRATE, and CPIAUCSL. Convert daily rates to monthly means and keep monthly series at monthly frequency. Return exactly 240 month rows from 2006-07 through 2026-06, preserve published missing values, and plot each series. Do not treat a reference-month date as its public release date.

DGS10: 10-year Treasury yield; DFF: federal funds rate; UNRATE: unemployment; CPIAUCSL: CPI.

Why it is written this way

- ▶ Exact series IDs remove ambiguity about the requested variables.
- ▶ A fixed calendar makes missing months visible.
- ▶ The release-date warning protects the forecast clock.

Let Codex Use AKShare, Then Inspect the Schema

Example instruction

Using AKShare, download China CPI year-on-year and official PMI. Keep 2006-07 through 2026-06, create a 240-month calendar, preserve missing months, and print the exact column names before selecting fields. Explain the units and release timing. Stop if the interface has changed.

- ▶ CPI and PMI are monthly; GDP is quarterly and should not be stretched into 240 fake monthly observations.
- ▶ For research, verify with official or licensed sources.

```
import akshare as ak
import matplotlib.pyplot as plt
cpi = ak.macro_china_cpi_yearly()
pmi = ak.macro_china_pmi()

print(cpi.columns.tolist())
print(pmi.tail())
cpi.to_csv("cpi.csv", index=False)
pmi.to_csv("pmi.csv", index=False)

# After checking the columns:
monthly = pmi.sort_values(pmi.columns[0])
monthly.plot(x=monthly.columns[0],
             y=monthly.columns[1])

plt.show()
```

Use Codex to Align One FRED Series

Example instruction

Use Python to download FRED series UNRATE from the public CSV interface. Keep observation months 2006-07 through 2026-06, preserve the observation-date field and published missing values, and report the schema, endpoints, row count, and units. Save the raw and cleaned files separately.

Expected object

- ▶ 240 monthly rows;
- ▶ an observation date and UNRATE value;
and
- ▶ no silent interpolation.

Do not confuse

- ▶ reference month with release date;
- ▶ current vintage with real-time vintage; or
- ▶ a missing release with a zero value.

Plot FRED, Then Check the Timing

Follow-up instruction

Run Python to plot UNRATE against its observation month. Then repeat for DGS10, DFF, and CPIAUCSL, using monthly means for the daily rate series. Return the chart, the aggregation rule, and a table of missing months.

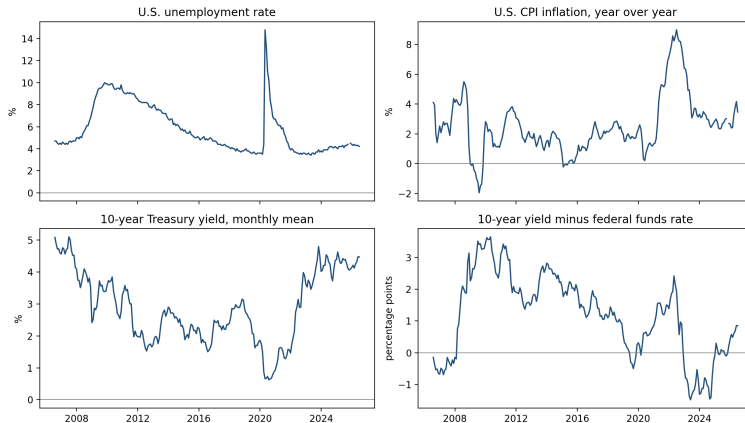
1. Does the chart use the requested variable and units?
2. Did daily rates become monthly means rather than month-end values?
3. Are gaps visible rather than silently filled?
4. What extra release lag is required before using each series for prediction?

Keep in mind

Python drew the chart. Now work out when each value could first have been seen.

Macro Variables Use the Same 240-Month Calendar

A common 240-month macro window: July 2006 to June 2026



Source: Federal Reserve Bank of St. Louis FRED; 240 month rows, 2006-07 through 2026-06, downloaded July 2026. Published missing values are retained rather than silently interpolated.

The Unemployment Observation Date Is Not the Release Date

Suppose a row is labeled 2026-06-01. That label identifies the reference month, not the day on which investors learned the unemployment rate.

- ▶ Monthly macro series are normally released after the reference period.
- ▶ Previously published values can be revised.
- ▶ A current FRED download may contain revised history rather than the vintage seen in real time.
- ▶ A predictive exercise therefore needs a release calendar and, when revisions matter, vintage data such as ALFRED.

Do not do this

Do not forward-fill a monthly series from the first day of its reference month and call it a real-time predictor.

Ask for Statements and Filing Dates Together

Company-data instruction

Using Python and `yfinance`, download AAPL quarterly income statements, balance sheets, cash flow statements, and SEC filing metadata. Keep total revenue, net income, total assets, stockholders' equity, and operating cash flow. Return period end and the first subsequent 10-Q or 10-K filing date. Print the exact row labels before selecting fields, and stop if a requested field is absent.

Why it is written this way

- ▶ Filing dates determine when financial values become usable.
- ▶ Printing row labels catches changes in the vendor schema.
- ▶ The stop rule prevents silent substitution of missing fields.

Let Codex Inspect One Financial Statement

```
import yfinance as yf

apple = yf.Ticker("AAPL")
income = apple.quarterly_income_stmt

print(income.index)
print(income.columns)
revenue = income.loc["Total Revenue"]
print(revenue)
```

Package output is not a stable database schema

Inspect the labels first. In a full application, add the balance sheet, cash flow statement, and filing dates only after confirming the fields.

Ask Codex to Plot One Financial-Statement Item

```
import matplotlib.pyplot as plt

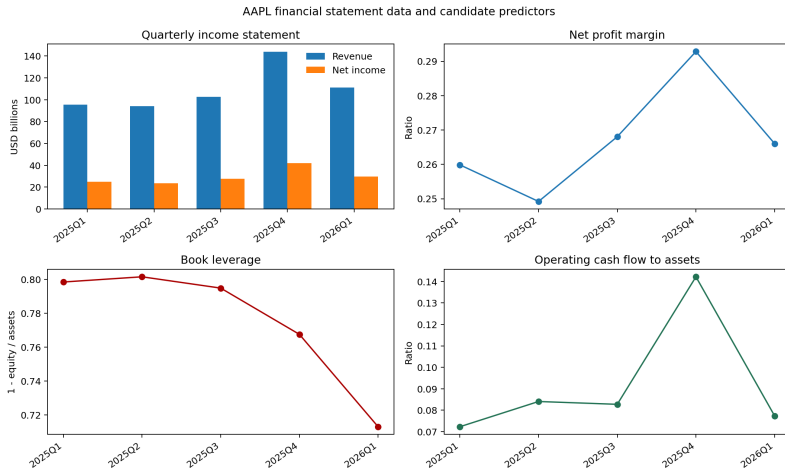
revenue = income.loc["Total Revenue"]
revenue = revenue.sort_index()

revenue.plot(kind="bar")
plt.title("AAPL quarterly revenue")
plt.show()
```

Keep in mind

Begin with one familiar item. The next step is to construct and plot several financial ratios.

Financial Statements Become Candidate Predictors



Source: Yahoo Finance through yfinance; illustrative download generated July 2026.

Turn Accounting Levels into Comparable Predictors

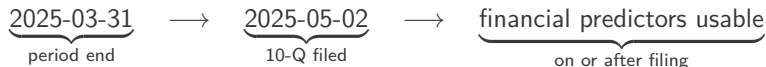
Predictor	Formula	Interpretation
Profit margin	net income / revenue	earnings produced per dollar of sales
Book leverage	$1 - \text{equity} / \text{assets}$	share of assets financed by non-equity claims
Asset growth	$\text{assets} / \text{lagged assets} - 1$	expansion of the accounting asset base
Cash flow to assets	operating cash flow / assets	operating cash generation relative to scale

Keep in mind

These are candidate features for teaching, not a claim that each one predicts AAPL's next-week active return.

Use the Filing Date, Not the Quarter End

For the statement period ending 2025-03-31:



- ▶ The values describe the quarter ending in March.
- ▶ Investors could not use the downloaded statement fields in March if they were filed in May.
- ▶ The predictor should remain missing until its filing date, then persist until the next filing.

Source for the illustrative filing date: Yahoo Finance SEC filing metadata, retrieved July 2026.

Specify the Date-Matching Rule in Plain Language

Date-matching instruction

For each trading date, attach only the latest financial statement whose filing date is on or before that trading date. Keep both the fiscal period end and filing date in the output. Do not match on the fiscal period end. Show three rows around every filing date so I can check when the values change.

- ▶ Before a filing date: the new statement must not appear.
- ▶ On or after a filing date: the new statement may appear.
- ▶ In an application, the code performs the merge; the student checks the rule.

Keep in mind

For now, understanding and checking the information rule matters more than memorizing the merge command.

One Weekly Stock Row Can Hold Three Information Types

Column	Source	Timing rule
<code>mom20d_rank</code>	daily adjusted prices	uses data through the prior week
<code>rv20d_rank</code>	daily returns	window ends before the forecast week
<code>profit_margin</code>	financial statement	available only after its filing date
<code>market_return</code>	SPY or market index	realized in forecast week t
<code>active_return</code>	stock and market returns	stock return minus market return

Keep in mind

The row is labelled by the forecast week; every predictor must already be known before that week's return begins.

Construct Predictors without Leakage

Fix the Weekly Stock Forecast Clock Before Writing Code

At the end of week t :

$$\underbrace{x_t}_{\text{known after week } t \text{ closes}} \longrightarrow \underbrace{r_{t+1} = P_{t+1}/P_t - 1}_{\text{realized during week } t+1}.$$

- ▶ The saved row is labelled by forecast week $t + 1$.
- ▶ Stock characteristics use prices and volume no later than week t .
- ▶ Filed accounting data may be carried forward; unfiled values remain unavailable.
- ▶ Download extra pre-sample days to build rolling signals, then keep exactly 156 forecast weeks.

Three Weekly Stock Predictors for the First Exercise

Variable	Formula known before week t	What the plot should reveal
4-week momentum	$P_{t-1}/P_{t-5} - 1$	short-horizon trend and reversal
4-week volatility	$\sqrt{52} \text{sd}(r_{t-4}, \dots, r_{t-1})$	changing weekly risk
4-week MA gap	$P_{t-1}/\bar{P}_{t-1,4} - 1$	distance from the recent trend
Weekly target	$P_t/P_{t-1} - 1$	noisy outcome and correct forecast direction

Keep in mind

Each of the three has a formula and a window. Write both down now, because Class 3 re-estimates them and the definitions must not move.

Use Codex to Construct One Predictor First

Example instruction

Using the checked weekly adjusted-price series, run Python to build four-week momentum known before each forecast week. Define the formula in plain language, lag it so that week t never uses week- t returns, and show three consecutive rows with every source price used in the calculation.

Python should return

- ▶ forecast-week label;
- ▶ four-week source prices;
- ▶ raw momentum; and
- ▶ lagged predictor.

Students verify

- ▶ no future price enters;
- ▶ the sign and magnitude are plausible; and
- ▶ the first usable week is correctly missing.

Check the Feature Table

```
price = aapl["Close"].resample("W-FRI").last()
return_1w = price.pct_change()
momentum_4 = price.pct_change(4).shift(1)
volatility_4 = return_1w.rolling(4).std().shift(1) * 52**0.5
target = return_1w

print(momentum_4.head())
```

Keep in mind

Codex may create one object per line so that the calculation is easy to check. In class, inspect the returned columns and timing rather than typing the syntax.

Never Trust a Lead or Lag without a Three-Row Check

Pick three consecutive forecast weeks and verify by hand:

Forecast week	Information allowed in predictors	Return stored as target
$t - 1$	values observable after week $t - 2$	$P_{t-1}/P_{t-2} - 1$
t	values observable after week $t - 1$	$P_t/P_{t-1} - 1$
$t + 1$	values observable after week t	$P_{t+1}/P_t - 1$

Common failure

An AI assistant may produce syntactically valid code with the target shifted in the wrong direction.

Five Ways Leakage Enters during Feature Construction

1. centering or scaling with the full sample before the train/test split;
2. using a centered rolling window that includes future observations;
3. aligning accounting data by fiscal period instead of release date;
4. filling a missing value with information observed later; and
5. selecting predictors after examining final test-period performance.

Keep in mind

A predictor formula is not complete until its information date is specified.

Four Plots, Four Different Questions

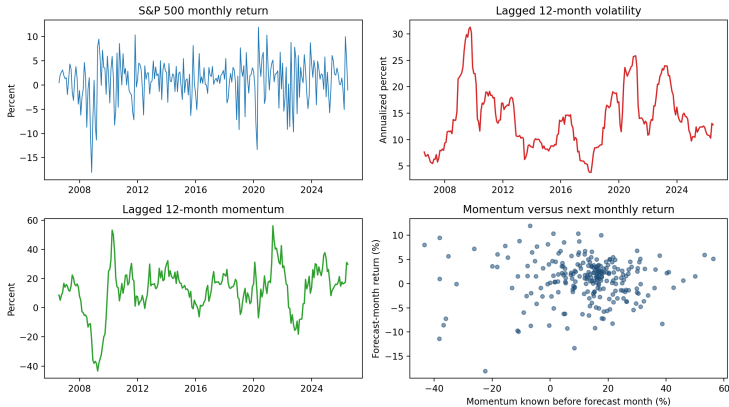
1. **Price through time:** Did the download cover the intended sample? Are adjustments plausible?
2. **Returns through time:** Are there extreme observations, gaps, or suspicious zeros?
3. **Rolling volatility:** Does risk vary through time as expected?
4. **Predictor versus target:** Is the relation visible, nonlinear, dominated by outliers, or simply weak?

Keep in mind

Each plot answers one question. Decide which question you are asking before you request the chart.

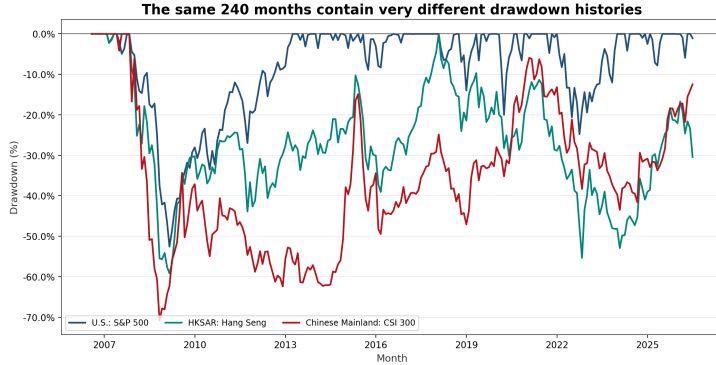
A 240-Month Market-Prediction Dashboard

The 240-month U.S. market-prediction table



Read across each row: compare the predictor's timing and scale with the return target before treating any visible pattern as forecast evidence.

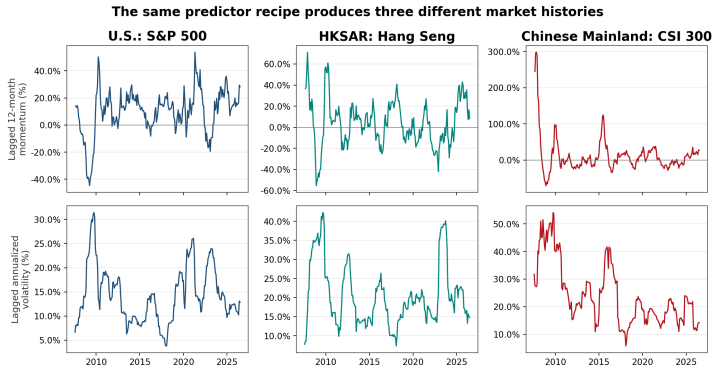
Price Growth Can Hide Very Different Drawdowns



Keep in mind

Two markets can reach the same cumulative return along very different paths. Drawdown is the part a client actually lives through.

The Same Predictor Recipe Produces Different Histories



Keep in mind

Momentum and volatility use the same code in all three markets, but their ranges and episodes differ. Always inspect before modeling.

Read an Explanation against the Evidence

Ask four questions:

- ▶ Which observations drive the axes and apparent pattern?
- ▶ Are the units and annualization correct?
- ▶ Did the rolling statistic use only past and current data?
- ▶ Should the scatter use raw, ranked, or winsorized variables?
- ▶ Does the conclusion identify a scale, benchmark, and forecast horizon?

The class chooses one task for a live run:

1. compare the weekly returns of two HSTECH 30 constituents while retaining the same 156 forecast weeks;
2. add one FRED series to the 240-month macro calendar and state its release-time problem;
3. construct one additional price-based predictor from the AAPL weekly series; or

Keep in mind

First agree on the sample, variable definition, dates, and checks. I then paste the prompt, run it, and we inspect the evidence together. You do not need to type Python in class.

If the Live Run Fails, Read the Evidence

Failure type	What to print before guessing	Usual cause
Network	the raw response or an empty frame	the source refused, throttled, or returned nothing
Ticker	the requested symbol and the returned index	a delisted, renamed, or wrongly padded code
Column	<code>df.columns</code>	a renamed field, or a multi-level frame
Date	the first and last index values and their type	unparsed strings, or two calendars joined
Type	<code>df.dtypes</code>	a numeric column arrived as text and silently blocked arithmetic
Logic	the object on the line before the failure	it runs, but computes something you did not specify

Change one input, rerun from the top, and record the fix. The task is to diagnose the evidence, not to guess Python syntax.

Whole-Class Check: Four Questions

Look at the live output and ask:

1. What does one row represent, and which date belongs to the predictor and target?
2. Are the prices adjusted, and how was that choice verified?
3. Does each macro or accounting value use its release or filing date?
4. Can the code regenerate the table and plot from a clean environment?

What to Remember

1. Define the sample, variables, and information date before using code.
2. Keep trading dates, macro releases, and filing dates separate; check one lead or lag by hand.
3. Use plots to diagnose the data before fitting a model.

Next Class: Regression and Return Predictability

From a plot to a forecast

We move from constructing and checking data to asking whether a variable improves a return forecast.

- ▶ Read a scatter plot and a fitted line without confusing association with prediction.
- ▶ Compare simple and multiple regression forecasts with a 12-month moving-average benchmark.
- ▶ Interpret forecast errors, out-of-sample performance, and stability across time.

The next question is not whether a pattern exists, but whether it improves a decision.